

Supplementary Material for *PSDissect: A CFG-Guided, Semantics-Preserving Interactive Deobfuscation Framework for PowerShell Scripts*

This document provides the six supplementary sections referenced by the main paper: additional obfuscation examples (S1), complete CFG construction examples (S2), the complete execution denylist (S3), the complete set of recoverable-fragment types (S4), the complete key-API list (S5), and the full residual obfuscation-burden definition (S6).

Contents

S1 Additional Obfuscation Examples	2
S2 Complete CFG Construction Examples	3
S2.1 Control Lowering	4
S2.2 Function Subgraph	9
S2.3 Runtime Expansion	10
S3 Complete Denylist	10
S4 Complete Recoverable-Fragment Types	13
S5 Complete Key-API List	16
S6 Full Residual Obfuscation-Burden Definition	21
S6.1 Feature Set	21
S6.2 Per-Feature Residual Ratio	21
S6.3 Sample-Level Residual Burden	21
S6.4 Pairwise Aggregation	22
S6.5 Weight-Sensitivity Analysis	23

S1 Additional Obfuscation Examples

The following listings provide additional benign obfuscation examples that preserve the behavior of Write-Output ("PSDissect").

```
1 # Token escapes, mixed case, and redundant spacing
2 Wr'ite-Out'put ( "PSDissect" )
3
4 # Substring extraction
5 Write-Output ('xxPSDissectyy'.Substring(2, 9))
6
7 # Replacement-based reconstruction
8 Write-Output ('PXDissect' -replace 'X','S' -replace 'Y','e')
9
10 # Reversed character sequence
11 $c = 'tcessiDSP'.ToCharArray(); [array]::Reverse($c)
12 Write-Output (-join $c)
13
14 # Alias and indirect command target
15 & (Get-Alias ('wr' + 'ite')).Name ('PS' + 'Dissect')
16
17 # BXOR decoding
18 $k = 23
19 $b = 71,68,83,126,100,100,114,116,99
20 Write-Output (-join ($b | ForEach-Object { [char]($_ -bxor $k) }))
21
22 # Decoder encapsulated in a function
23 function Decode-Text($x, $k) {
24     -join ($x | ForEach-Object { [char]($_ -bxor $k) })
25 }
26 & ('Write-' + 'Output') (Decode-Text $b $k)
```

Listing 1: Additional PowerShell obfuscation techniques applied to the benign command.

```
1 #Token
2 &("{1}{0}{2}" -f 'e-', 'Writ', 'Output') ((("{0}{1}" -f 'PSDi', 'ssect'))
3
4 # String
5 &((([String]$vERBOSepREfeRenCe)[1,3]+'x'-Join')) ((('Writ'+ 'e-Out'+ 'put ' + '{0}'+ 'PSD'+
    issect{'+'0}')) -f [Char]34))
6
7 $LGzV1j = " )43]Rahc[]GNIRts[, )911]Rahc[+17]Rahc[+05]Rahc[(ECALPeR.)')wG2'+ 't'+ 'cessi'+ 'D
    '+ 'SP'+ 'wG2( t'+ 'uptu'+ '0-etirW'( ( noISSErPx-eKoVNI";. ( $shellId[1]+$shellId[13]+'X') (
    -join $lgzv1j[ - 1..-( $lgzv1j.lEngTH ) ] )
```

Listing 2: Token-, string-, and reversal-based obfuscation generated from the benign command.

```
9 # ASCII
10 -Join(( 87,114,105 , 116,101 ,45 , 79, 117, 116,112 ,117 , 116, 32, 40 , 34, 80 ,83
    ,68,105,115, 115, 101, 99 ,116 ,34,41 )|ForEach{ ( [int]$_-as[char] ) } )|&($shellId[1]+$
    ShellId[13]+'X')
11 # Hex
12 iEX( -join( ( 57 ,72,69,74,65,'2d', '4f' , 75,74 ,70 ,75 ,74,20,28, 22,50, 53 , 44,69 ,73, 73
    , 65 ,63,74, 22,29 )|ForEach{ ( [COnVerT]::ToINT16( ([string]$_) ,16 )-as[CHAR] ) } ) )
13 # Binary
14 .( $shellId[1]+$shellId[13]+'X') ([string]::Join('' ,(( 1010111, 1110010, 1101001 , 1110100 ,
    1100101 , 101101 , 1001111 , 1110101 , 1110100,1110000 , 1110101,1110100,100000 , 101000
    ,100010 ,1010000,1010011, 1000100, 1101001 ,110011,1110011 ,1100101 , 1100011 ,
    1110100,100010,101001)|FoREaCH-oBJeCt{( [cOnVerT]::tOInt16(( [string]$_) , 2) -as[chAr] ) } )
    ) )
```

Listing 3: Decimal, hexadecimal, and binary encodings of the benign command.

```

19 # Compress
20 & ( $vERbosEprefeRencE.tOSTriNg() [1,3]+'x'-JoIn'' ) (NeW-OBjeCT SyStEM.io.StREamrEaDER( (
    NeW-OBjeCT SYSTem.io.cOmPRessIoN.DeFlaTeStReAM([SyStEM.iO.MEmoRysTream][cOnvErT]::
    FrOMbaSE64strinG( 'Cy/KLEnV9S8tKSgtUdBBQCgh2ySwuTk0uUdIEAA=='),[SyStEM.io.CompReSSioN.
    cOmpressioNmODE]::DEcoMprEsS )) , [tExT.enCOdiNG]::AScii)).reADToend( )

```

Listing 4: Compressed-loader obfuscation of the benign command.

S2 Complete CFG Construction Examples

Table 1 enumerates the PowerShell construct families handled explicitly by CFG construction. AST forms that use the same construction rule are grouped in one row. The complete examples that follow show the resulting CFGs for the principal non-sequential structures: conditionals, loops, switches, exception handling, functions, and runtime-generated code.

Table 1: PowerShell construct families and their CFG construction rules.

Construct family	AST forms / examples	CFG construction rule
Sequential statements	Assignments, command expressions, and single commands	Each statement is represented by one CFG node and connected to the preceding and following nodes by sequential edges. Nested expressions remain associated with that node as recoverable fragments.
Parameters and named blocks	Script/function parameters; begin , process , and end blocks	Parameter binding and named-block boundaries are represented explicitly so that later nodes execute with the correct input and local bindings.
Conditional branches	if , elseif , and else	Each predicate becomes a condition node with explicit true and false edges. Branch tails converge at a merge node unless control leaves through return , throw , or exit .
Switch dispatch	switch clauses and default	The switch input is stored once, each clause is modeled as a comparison-and-dispatch branch, and unmatched input reaches the default path.
Loops	for , foreach , while , do-while , and do-until	Loop initialization, condition checking, body execution, and iteration are represented separately. GUID-based internal variables materialize collection and index state where PowerShell leaves that state implicit.
Exception handling	try , typed or untyped catch , and finally	Protected, handler, and cleanup regions receive explicit edges. Normal and exceptional exits are routed through finally when it is present.

Construct family	AST forms / examples	CFG construction rule
Pipelines	Multi-element pipelines and pipeline-valued assignments	Pipeline elements are split into ordered nodes. GUID-based internal variables carry intermediate objects between elements while retaining each element's source offsets.
Pipeline script blocks	<code>ForEach-Object</code> , <code>Where-Object</code> , and <code>Select-Object</code>	Supported pipeline cmdlets receive dedicated execution handling. Their script blocks are represented as callable subgraphs, and per-input invocation instances are recorded.
Functions	Function definitions and calls	A definition creates a separate function subgraph with parameter, start, body, and end nodes. A call site records its invocation context, enters the subgraph, and resumes the caller after return.
Script blocks	Literal script blocks, script-block variables, call operator, dot sourcing, and supported invocation methods	Each executable script block is assigned a GUID-based identity and a separate subgraph. Variable-to-subgraph mappings allow invocation sites to enter the corresponding body while preserving arguments and call-site identity.
Control transfer	<code>return</code> , <code>break</code> , <code>continue</code> , <code>throw</code> , and <code>exit</code>	Dedicated nodes connect directly to the correct function, loop, switch, exception, or script exit target rather than to the next sequential statement.
Runtime-generated code	<code>Invoke-Expression</code> , <code>ScriptBlock::Create</code> , <code>NewScriptBlock</code> , PowerShell host commands, and wrapped dynamic invocations	Once generated source text is recovered, PSDissect parses it into a new AST, appends a runtime-generated subgraph, links it to the invocation site, and recursively applies the same construction rules.

S2.1 Control Lowering

CFG Example 1: `if/elseif/else`

This conditional is lowered into chained condition nodes with explicit true and false successors. The final `else` block becomes the fallback path after the preceding tests fail.

Listing 5: Original script for CFG Example 1.

```

1 $value = 2
2
3 if ($value -eq 1) {
4     Write-Output 'one'
5 }
6 elseif ($value -eq 2) {
7     Write-Output 'two'
8 }
9 else {
10    Write-Output 'other'

```

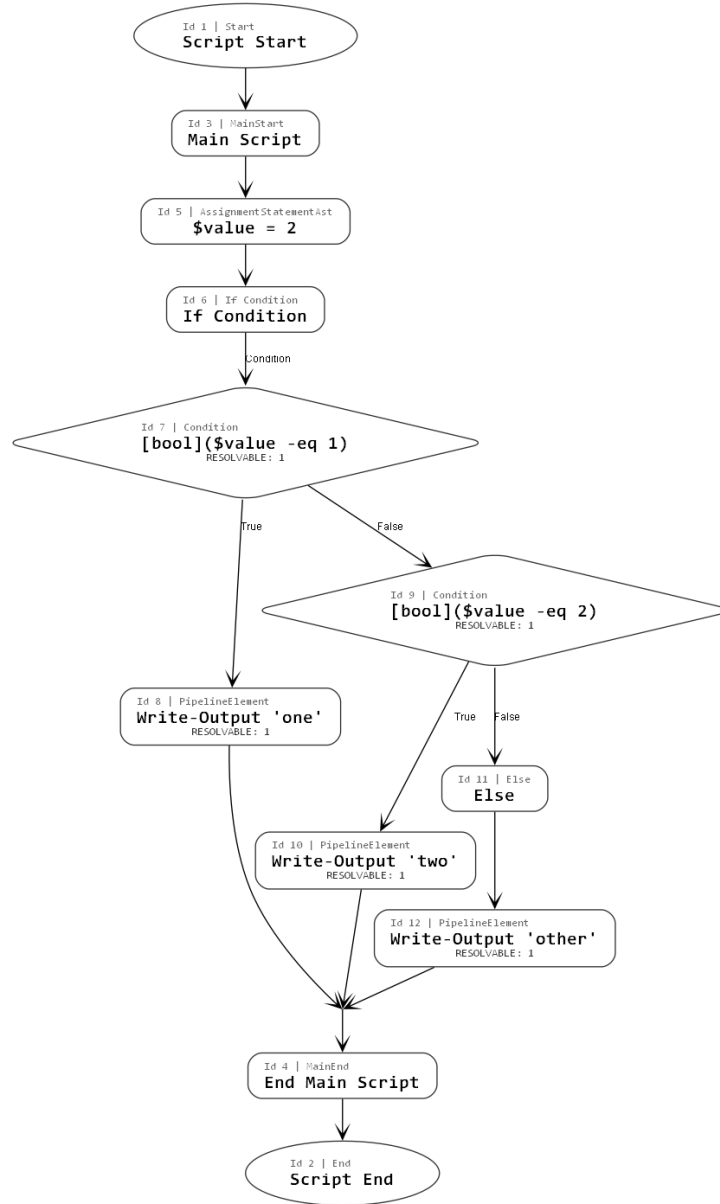


Figure 1: CFG for CFG Example 1.

CFG Example 2: foreach

This loop is lowered into explicit initialization, condition, body, and iteration steps. Because the iteration state is not fully explicit in the source text, PSDissect materializes internal GUID-based variables during lowering to avoid conflicts with user-defined variables.

Listing 6: Original script for CFG Example 2.

```
1 $items = @('a', 'b')
2
3 foreach ($item in $items) {
4     Write-Output $item
5 }
```

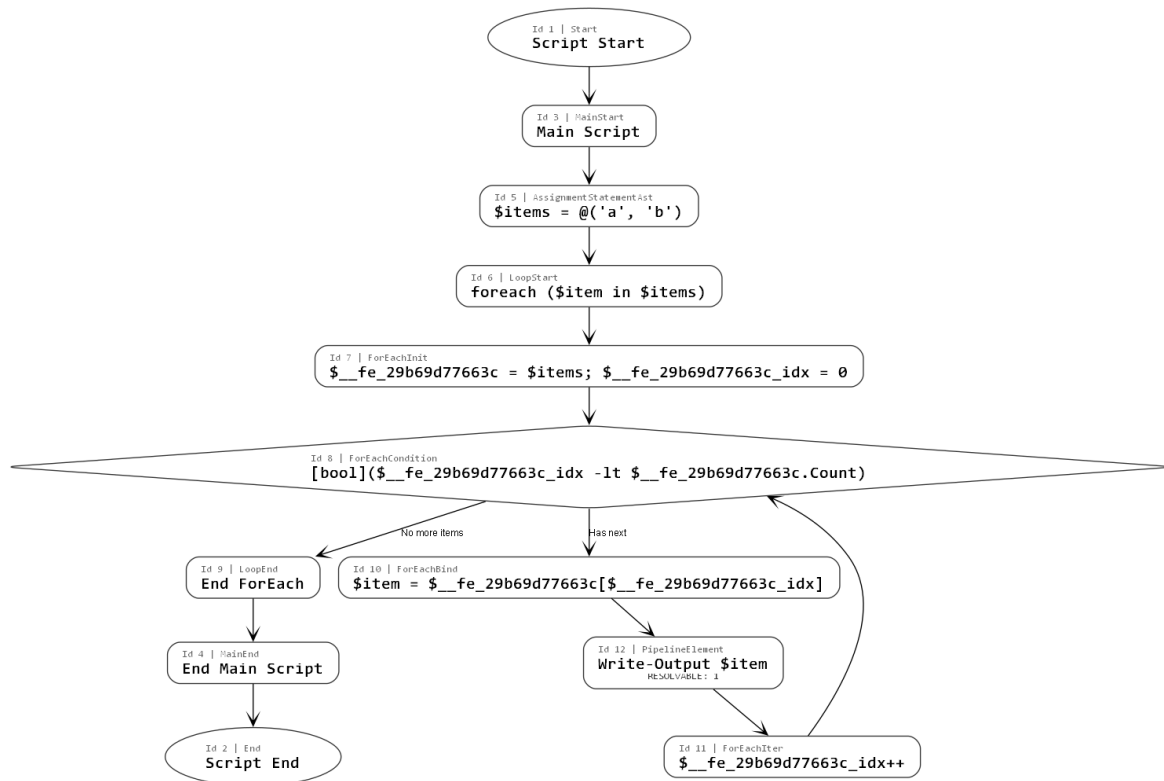


Figure 2: CFG for CFG Example 2.

CFG Example 3: switch

The **switch** statement is lowered into explicit comparison-and-dispatch paths. Each case arm is represented as a separate control-flow branch, while the **default** arm is modeled as the final fallback path.

Listing 7: Original script for CFG Example 3.

```

1 $kind = 'b'
2
3 switch ($kind) {
4     'a' { Write-Output 'alpha' }
5     'b' { Write-Output 'beta' }
6     default { Write-Output 'other' }
7 }

```

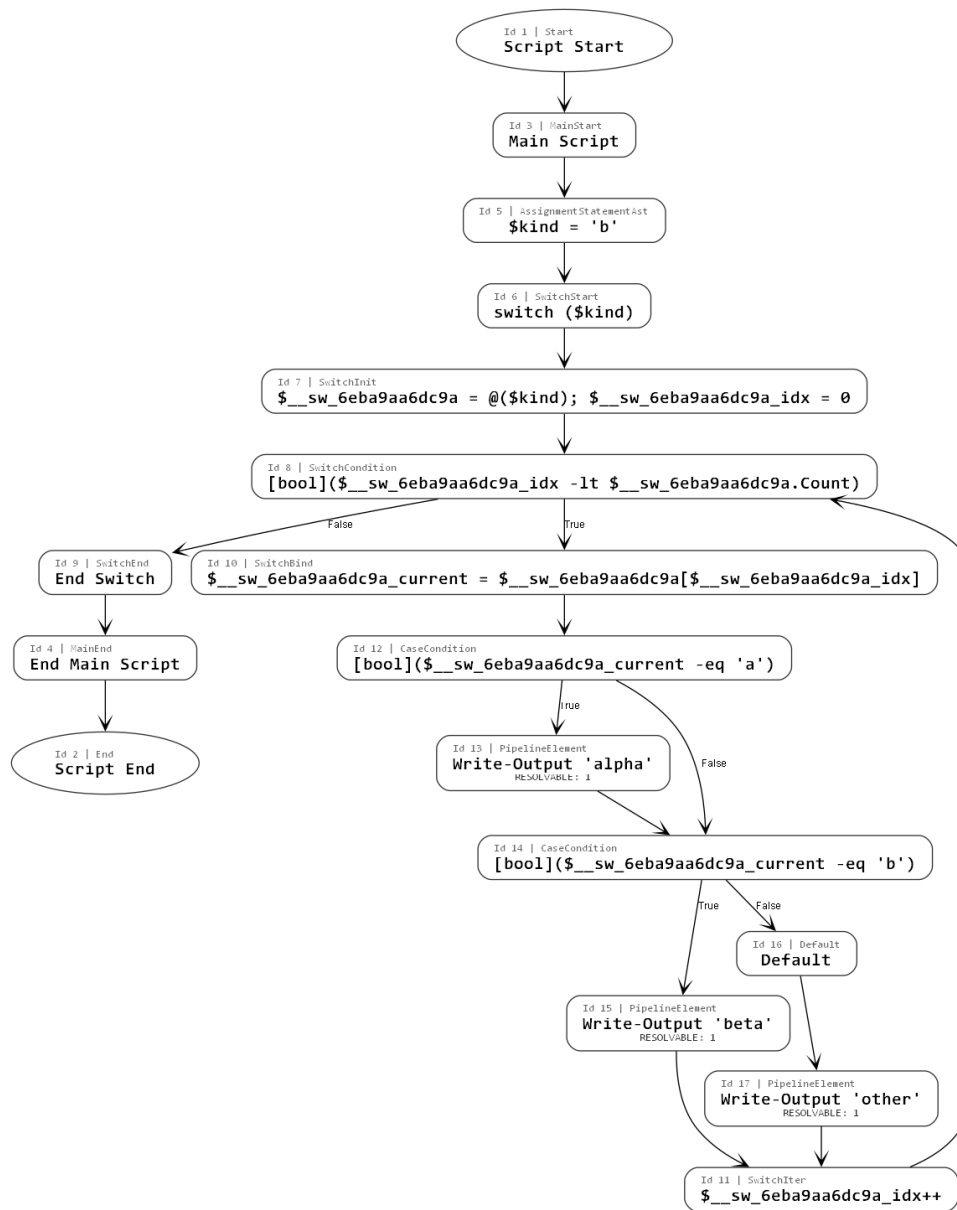


Figure 3: CFG for CFG Example 3.

CFG Example 4: try/catch/finally

This structure is lowered by making exception-handling control flow explicit. The CFG represents entry into the protected region, transfer into the `catch` branch when an exception is raised, and mandatory progression through the `finally` block.

Listing 8: Original script for CFG Example 4.

```
1 try {  
2   throw 'boom'  
3 }  
4 catch {  
5   Write-Output 'handled'  
6 }  
7 finally {  
8   Write-Output 'cleanup'  
9 }
```

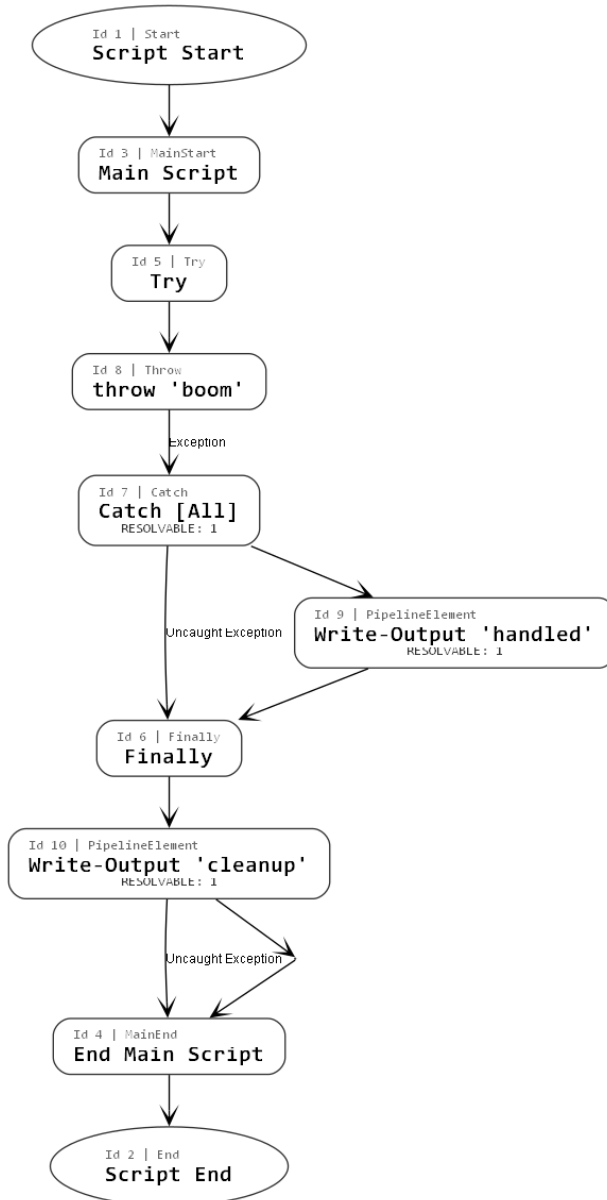


Figure 4: CFG for CFG Example 4.

S2.2 Function Subgraph

CFG Example 5: function

The function body is lowered into a separate subgraph rather than being inlined into the caller's top-level sequence. The call site in the main graph retains invocation metadata, allowing controlled execution to enter the callee and later resume the caller within the same runspace context.

Listing 9: Original script for CFG Example 5.

```

1 function Get-Greeting {
2     param(
3         [string]$Name
4     )
5
6     return "Hello, $Name"
7 }
8
9 $result = Get-Greeting -Name 'CFG'
10 Write-Output $result

```

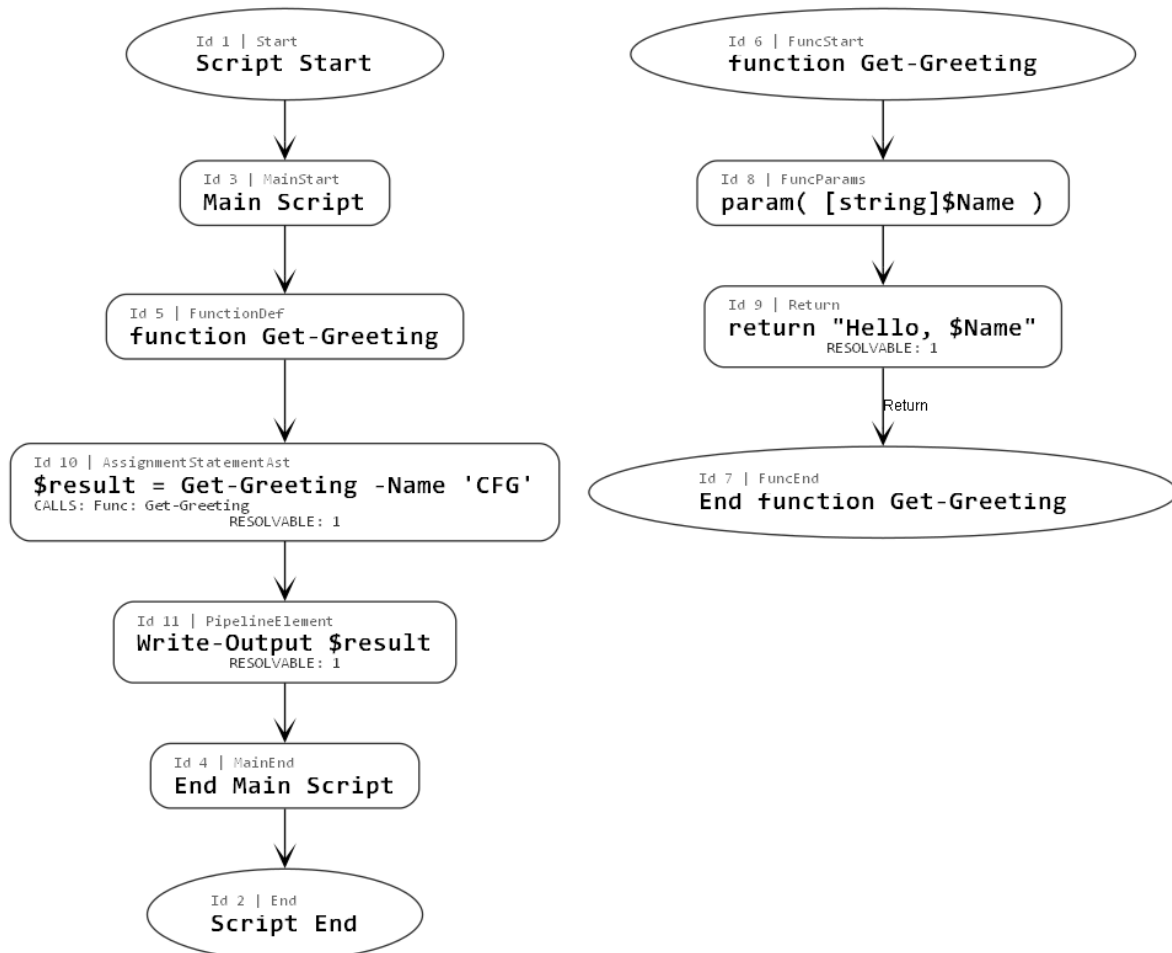


Figure 5: CFG for CFG Example 5.

S2.3 Runtime Expansion

CFG Example 6: iex

The initial CFG contains the dynamic invocation site and the string-valued code carrier. Once the argument of `iex` is recovered at runtime, PSDissect parses the generated code and appends a runtime-generated subgraph linked to the original invocation site.

Listing 10: Original script for CFG Example 6.

```
1 $code = '$message = ''dynamic''; Write-Output $message'
2 iex $code
```

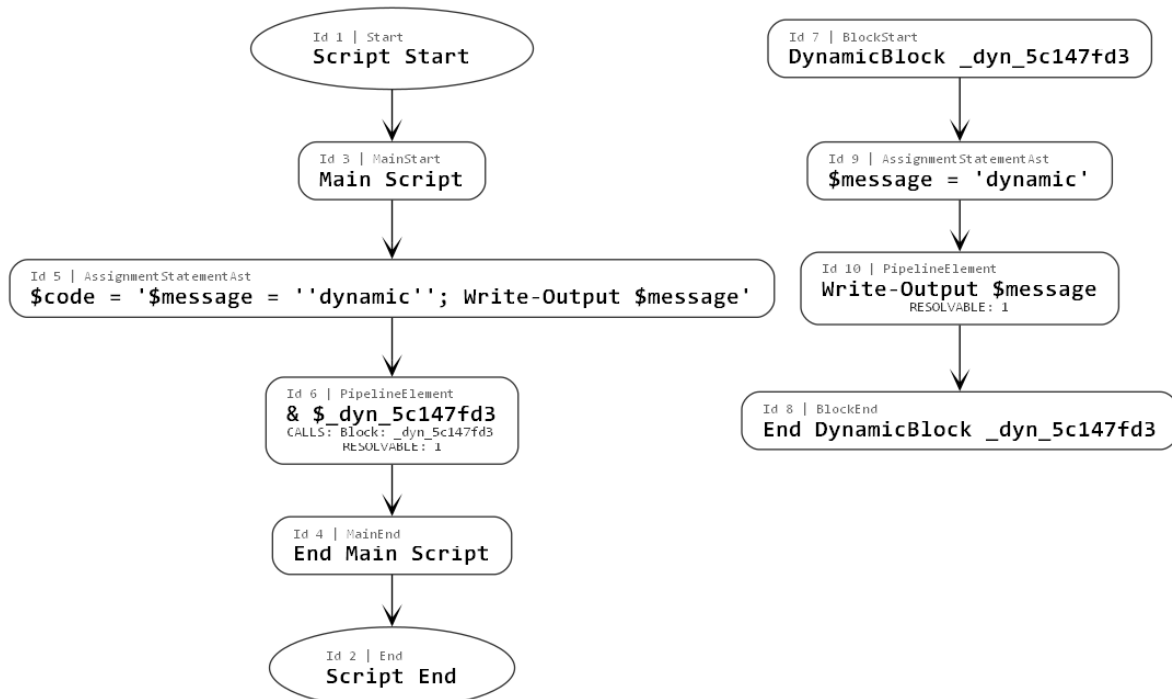


Figure 6: CFG for CFG Example 6.

S3 Complete Denylist

PSDissect enforces layered safety constraints during controlled execution. Listing 11 gives the complete command-level `ForbiddenCommands` set, the additional .NET and process-launch checks, and the pattern-based pre-execution stop indicators implemented in `Execute-CFG.ps1`. A command or method matched by the execution denylist is blocked and represented by a placeholder so that CFG traversal can continue. A payload matched by a pre-execution stop indicator is not recursively executed; PSDissect instead retains the recovery results obtained before that boundary.

Listing 11: Complete PSDissect execution denylist and pre-execution stop indicators.

```
1 [Command-level forbidden commands]
2
3 File deletion / clearing:
4 Remove-Item, del, rm, rmdir, rd, ri, erase
5 Clear-Content, clc
6 Clear-ItemProperty, clp
7 Remove-ItemProperty, rp
8 Clear-RecycleBin
```

```

9
10 File move / rename:
11     Move-Item, move, mv, mi
12     Rename-Item, ren, rni
13
14 Network operations:
15     Invoke-WebRequest, iwr, curl, wget
16     Invoke-RestMethod, irm
17     Start-BitsTransfer
18     Add-BitsFile
19     Complete-BitsTransfer
20     Test-NetConnection
21     Test-Connection, ping
22     Send-MailMessage
23
24 Process launch / waiting:
25     Start-Process, start, saps
26     Wait-Process
27     Debug-Process
28
29 System control:
30     Stop-Computer
31     Restart-Computer
32     Suspend-Computer
33     Checkpoint-Computer
34     Restore-Computer
35
36 Remote execution / remoting:
37     Invoke-Command, icm
38     Enter-PSSession
39     New-PSSession
40     Enable-PSRemoting
41     Disable-PSRemoting
42     Enable-PSSessionConfiguration
43     Register-PSSessionConfiguration
44
45 User interaction:
46     Read-Host
47     Get-Credential
48     Out-GridView
49
50 Long waits:
51     Wait-Event
52     Wait-Job
53
54 Disk / archive operations:
55     Format-Volume
56     Clear-Disk
57     Initialize-Disk
58     Set-Disk
59     Remove-Partition
60     Optimize-Volume
61     Expand-Archive
62     Compress-Archive
63
64 Other hanging side effects:
65     Out-Printer
66     Start-Transcript
67     Stop-Transcript
68
69 [Additional command-level rules]
70
71 COM object creation:

```

```

72 New-Object -ComObject / -Com
73
74 External executable launch:
75 *.exe, *.com, *.bat, *.cmd, *.vbs, *.js, *.wsf, *.msi, *.hta
76
77 [Dangerous .NET method patterns]
78
79 File deletion:
80 System.IO.File::{Delete}
81 System.IO.Directory::{Delete}
82 System.IO.FileInfo::{Delete}
83 System.IO.DirectoryInfo::{Delete}
84
85 File move:
86 System.IO.File::{Move}
87 System.IO.Directory::{Move}
88 System.IO.FileInfo::{MoveTo}
89 System.IO.DirectoryInfo::{MoveTo}
90
91 Network access:
92 System.Net.WebClient::{
93     DownloadFile, DownloadFileAsync,
94     DownloadData, DownloadDataAsync,
95     DownloadString, DownloadStringAsync,
96     UploadFile, UploadFileAsync,
97     UploadData, UploadDataAsync,
98     UploadString, UploadStringAsync,
99     UploadValues, UploadValuesAsync,
100     OpenRead, OpenReadAsync,
101     OpenWrite, OpenWriteAsync
102 }
103 System.Net.HttpWebRequest::{
104     GetResponse, GetResponseAsync,
105     GetRequestStream, GetRequestStreamAsync
106 }
107 System.Net.HttpWebResponse::{GetResponseStream}
108 System.Net.Sockets.TcpClient::{Connect, ConnectAsync, GetStream}
109 System.Net.Sockets.TcpListener::{
110     Start, AcceptTcpClient, AcceptTcpClientAsync,
111     AcceptSocket, AcceptSocketAsync
112 }
113 System.Net.Sockets.Socket::{
114     Connect, ConnectAsync,
115     Send, SendAsync, SendTo, SendToAsync,
116     Receive, ReceiveAsync, ReceiveFrom, ReceiveFromAsync,
117     Bind, Listen, Accept, AcceptAsync
118 }
119 System.Net.Sockets.UdpClient::{Connect, Send, SendAsync, Receive, ReceiveAsync}
120 System.Net.Dns::{
121     GetHostEntry, GetHostEntryAsync,
122     GetHostAddresses, GetHostAddressesAsync
123 }
124 System.Net.Mail.SmtpClient::{Send, SendAsync}
125
126 Process launch / waiting:
127 System.Diagnostics.Process::{Start, WaitForExit, WaitForExitAsync}
128
129 Archive extraction / GUI interaction:
130 System.IO.Compression.ZipFile::{ExtractToDirectory}
131 System.Windows.Forms.Form::{ShowDialog}
132
133 Thread / task waiting:
134 System.Threading.Thread::{Sleep, Join}

```

```

135     System.Threading.Tasks.Task::{Wait, WaitAll, WaitAny}
136
137 [Dangerous instance-method names]
138
139 If runtime type resolution succeeds and the invoked object matches a dangerous
140 type pattern, PSDissect also blocks instance methods with names such as:
141     Delete, MoveTo
142     DownloadFile, DownloadData, DownloadString
143     UploadFile, UploadData, UploadString
144     Connect, Send, Receive, GetStream
145     Start, WaitForExit
146     Sleep, Join, Wait
147     ShellExecute, Popup, ShowDialog, ExtractToDirectory
148
149 [Pre-execution stop indicators]
150
151 Delay and non-termination patterns:
152     Start-Sleep, sleep
153     System.Threading.Thread::Sleep
154     ping -n
155     while ($true), while (1), for (;;)
156
157 Interactive COM and shell automation:
158     WScript.Shell.Popup
159     Shell.Application.ShellExecute
160     COM ShellExecute, Run, or Exec
161     InternetExplorer.Application / Navigate
162
163 GUI payload patterns:
164     System.Windows.Forms, System.Drawing
165     Form creation, ShowDialog, Show
166     Add_Shown, Add_Click, Add_KeyDown
167
168 Archive extraction patterns:
169     Expand-Archive
170     System.IO.Compression.ZipFile::ExtractToDirectory
171     Shell CopyHere
172
173 Compressed-loader combinations:
174     Base64 decoding combined with DeflateStream or GzipStream
175     Base64 decoding combined with a memory-stream reader and dynamic execution

```

S4 Complete Recoverable-Fragment Types

PSDissect associates every recoverable fragment with a CFG node, its original source range, and the execution or invocation instance from which its value was obtained. Table 2 lists all fragment families that directly produce recovery records. The implementation may later assign reconstruction provenance such as `Resolvable`, `VariableRead`, or `DynamicInvoke`; these provenance labels are not additional PowerShell syntax types.

Table 2: Complete recoverable-fragment families supported by PSDissect.

Fragment family	Implementation type or record	Recovered source region and result
Binary expression	Binary	A complete BinaryExpressionAst , including arithmetic, comparison, formatting, concatenation, join/split, and other binary-operator expressions whose value can be evaluated without blocked dependencies.
Unary expression	Unary	A complete UnaryExpressionAst . It is identified during CFG construction; side-effecting forms are not independently evaluated, while safe context-independent forms may be recovered statically or through an enclosing expression.
Method invocation	MemberInvoke	A complete static or instance InvokeMemberExpressionAst . Calls with implicit side effects, blocked taint, or denylisted methods are excluded from ordinary evaluation.
Type conversion	Convert	A complete ConvertExpressionAst , such as casts to <code>[char]</code> , <code>[string]</code> , or a numeric type.
Expandable string	ExpandableString	A double-quoted or here-string expression whose interpolated variables and subexpressions can be resolved in the current execution context.
Index expression	Index	A complete indexing operation, including array, string, and member-result indexing.
Subexpression	SubExpression	A complete <code>\$(...)</code> expression evaluated in the current scope.
Member access	Member	A complete property or field access represented by MemberExpressionAst .
Parenthesized expression	Paren	A complete parenthesized pipeline or expression. The whole range is retained so that replacement cannot modify only an internal token.
Command expression	Command	A complete CommandAst . Commands are not passed to the generic expression evaluator; they are handled by the command-target, literalization, sensitive-sink, function, script-block, or dynamic-invocation records below.
Variable read	VariableRead Results	Each variable reference in a node's VarsRead set. Its current binding is read from the shared Runspace before the node executes and is associated with that reference's source offsets.
Resolved command target	CommandName	Alias, variable-based, concatenated, indexed, or recognized wildcard command targets that resolve with sufficient confidence. PSDissect records the complete command-target source range and canonical command name.

Fragment family	Implementation type or record	Recovered source region and result
Command-target assignment	<code>CommandTarget</code> <code>AssignmentResults</code>	The right-hand side of a variable assignment later used as a high-confidence indirect command target, e.g., the assignment that supplies the target of <code>& \$cmd</code> .
Canonical indirect invocation	<code>CanonicalCommand</code> <code>InvocationResults</code>	A complete call-operator command whose non-direct target is resolved with high confidence. The invocation operator, resolved target, and original arguments are retained as one replaceable region.
Assignment right-hand side	<code>AssignmentRhs</code>	The complete right-hand side of an assignment whose value is produced by a script-block invocation or materialized script block. This preserves script blocks as scalar values rather than replacing only an internal expression.
Literalizable command result	<code>Literalized</code> <code>CommandResults</code>	The right-hand side of supported deterministic environment queries assigned to a variable: <code>whoami, hostname, ipconfig /all Out-String, and Get-WmiObject/gwmi -Class Win32_ComputerSystem Out-String.</code>
Sensitive command argument	<code>SensitiveSink</code> <code>Results</code>	Complete recoverable arguments carrying URLs, launcher arguments, file or directory paths, registry paths, or host names in supported network, process, file, registry, and lookup commands. Blocked commands are not executed merely to obtain these values.
Function invocation	<code>FunctionCall</code> <code>Instances;</code> <code>FunctionInvoke</code> <code>Results</code>	Each function call site, its positional and named arguments, invocation identity, specialized body, and recoverable return value. Repeated calls at the same source position remain separate invocation instances.
Script-block value and invocation	<code>ScriptBlockCall</code> <code>Instances;</code> <code>ScriptBlock</code> <code>InvocationResults</code>	Script blocks created literally or at runtime, plus supported invocation sites through the call operator, dot sourcing, invocation methods, and script-block command parameters. The body, arguments, invocation identity, and call-site range are retained.
Pipeline process-block invocation	<code>ForEachObject</code> <code>ProcessIterations</code>	Each supported <code>ForEach-Object</code> process-block execution, including its input object, bound automatic variables, output, and iteration identity.

Fragment family	Implementation type or record	Recovered source region and result
Dynamic invocation	DynamicInvoke Results	Complete invocation sites for <code>Invoke-Expression/iex</code> , <code>ScriptBlock::Create</code> , <code>NewScriptBlock</code> , PowerShell host-command payloads, <code>AddScript</code> , and supported wrapped forms. Generated source text, the runtime-generated subgraph, return values, and invocation-instance information are recorded.
Context-independent fallback	static recovery records	For CFG paths that are not executed, PSDissect retains only source-local values that do not depend on runtime state, including literal strings, Boolean and numeric expressions, encoded-command text, and statically decoded script text.

After evaluation, a fragment result is retained only if it belongs to one of the materializable value classes in Table 3. Arrays are accepted recursively: every element must itself be recoverable. Unsupported runtime objects, values carrying blocked-command taint, and results that cannot be deterministically serialized as PowerShell source are discarded.

Table 3: Value classes retained after recoverable-fragment evaluation.

Retained value class	PowerShell/.NET types
Null	<code>\$null</code> .
Text and characters	<code>System.String</code> and <code>System.Char</code> .
Boolean	<code>System.Boolean</code> .
Integral numeric scalars	<code>Byte</code> , <code>SByte</code> , <code>Int16</code> , <code>UInt16</code> , <code>Int32</code> , <code>UInt32</code> , <code>Int64</code> , and <code>UInt64</code> .
Floating-point scalars	<code>Single</code> , <code>Double</code> , and <code>Decimal</code> .
Script block	<code>System.Management.Automation.ScriptBlock</code> , serialized as an explicit <code>[scriptblock] {...}</code> scalar expression.
Arrays and execution sequences	Arrays whose elements recursively belong to this table, including <code>char[]</code> and <code>byte[]</code> ; execution-result sequence containers are normalized before this check.
Blocked-operation placeholder	PSDissect’s dedicated blocked-command placeholder, retained only to propagate unavailable-value state and prevent dependent fragments from being treated as valid plaintext.

S5 Complete Key-API List

This section presents the full important-API configuration used in semantic-consistency evaluation. The file `fig/Important-API.json` contains 36 PowerShell cmdlets, 86 .NET APIs, and 11 native commands as positive entries, together with 33 exact alias rewrites, 58 capability-level aliases, and 11 exact noise APIs plus 5 noise prefixes.

The configuration has four parts: positive API names, exact alias rewrites, capability-level normalization rules, and noise filters. Concretely, these correspond to `cmdlets`, `dotNetApis`,

nativeCommands, apiAliases, capabilityAliases, noiseApis, and noiseApiPrefixes. The full configuration is shown in Listing 12.

Listing 12: Full important-API configuration used in semantic-consistency evaluation.

```
1 {
2   "sourceImage": "Important-API.png",
3   "counts": {
4     "cmdlets": 36,
5     "dotNetApis": 86,
6     "nativeCommands": 11,
7     "noiseApis": 11,
8     "noiseApiPrefixes": 5,
9     "apiAliases": 33,
10    "capabilityNames": 11,
11    "capabilityAliases": 58
12  },
13  "cmdlets": [
14    "add-content",
15    "add-type",
16    "clear-history",
17    "clear-recyclebin",
18    "copy-item",
19    "get-wmiobject",
20    "invoke-command",
21    "invoke-expression",
22    "invoke-item",
23    "invoke-restmethod",
24    "invoke-webrequest",
25    "move-item",
26    "new-item",
27    "new-itemproperty",
28    "out-file",
29    "receive-job",
30    "register-objectevent",
31    "remove-item",
32    "remove-itemproperty",
33    "rename-item",
34    "send-mailmessage",
35    "set-acl",
36    "set-content",
37    "set-executionpolicy",
38    "set-itemproperty",
39    "set-service",
40    "start-job",
41    "start-process",
42    "start-service",
43    "stop-process",
44    "stop-service",
45    "stop-transcript",
46    "test-connection",
47    "unregister-event",
48    "wait-job",
49    "wait-process"
50  ],
51  "dotNetApis": [
52    "AmsiBypass",
53    "AMSIBypa.Bypass",
54    "amsiUtils.Bypass",
55    "BP.AMS.Disable",
56    "bypass.AMSI.Disable",
57    "CompileAssemblyFromSource",
58    "CreateRemoteThread",
59    "Injector.Shellcode.Exec",
60    "Microsoft.Win32.RegistryKey.SetValue",
61    "Microsoft.Win32.RegistryKey.SetAccessControl",
62    "Microsoft.Win32.RegistryKey.CreateSubKey",
63    "OpenProcess",
64    "ProcessStarter.StartProcess",
65    "Shell.Application.CopyHere",
66    "Shell.Application.NameSpace",
67    "System.Diagnostics.Process.Start",
68    "System.IO.Directory.CreateDirectory",
```

```

69     "System.IO.File.WriteAllBytes",
70     "System.IO.File.WriteAllLines",
71     "System.IO.File.WriteAllText",
72     "System.Management.Automation.CommandInvocationIntrinsics.NewScriptBlock",
73     "System.Management.Automation.PowerShell.AddArgument",
74     "System.Management.Automation.PowerShell.AddParameter",
75     "System.Management.Automation.PowerShell.AddScript",
76     "System.Management.Automation.PowerShell.BeginInvoke",
77     "System.Management.Automation.PowerShell.Dispose",
78     "System.Management.Automation.PowerShell.EndInvoke",
79     "System.Management.Automation.PowerShell.Invoke",
80     "System.Net.CookieContainer.Add",
81     "System.Net.HttpListener.Start",
82     "System.Net.HttpListenerPrefixCollection.Add",
83     "System.Net.HttpWebRequest.GetRequestStream",
84     "System.Net.HttpWebRequest.set_Timeout",
85     "System.Net.RequestStream.Close",
86     "System.Net.RequestStream.Dispose",
87     "System.Net.RequestStream.Flush",
88     "System.Net.RequestStream.Write",
89     "System.Net.Sockets.NetworkStream.Flush",
90     "System.Net.Sockets.NetworkStream.Read",
91     "System.Net.Sockets.NetworkStream.Write",
92     "System.Net.Sockets.TcpClient.Connect",
93     "System.Net.Sockets.TcpClient.GetStream",
94     "System.Net.Sockets.TcpListener.Start",
95     "System.Net.WebClient.DownloadData",
96     "System.Net.WebClient.DownloadFile",
97     "System.Net.WebClient.DownloadString",
98     "System.Net.WebClient.UploadData",
99     "System.Net.WebClient.UploadFile",
100    "System.Net.WebClient.UploadString",
101    "System.Net.WebClient.UploadValues",
102    "System.Net.WebHeaderCollection.Add",
103    "System.Net.WebRequest.Create",
104    "System.Net.WebRequest.CreateHttp",
105    "System.Reflection.Assembly.Load",
106    "System.Reflection.Assembly.LoadFile",
107    "System.Reflection.Assembly.LoadFrom",
108    "System.Reflection.RtFieldInfo.SetValue",
109    "System.Runtime.InteropServices.Marshal.Copy",
110    "System.Runtime.InteropServices.Marshal.WriteByte",
111    "System.Security.AccessControl.RegistrySecurity.SetAccessRule",
112    "System.Security.AccessControl.RegistrySecurity.SetOwner",
113    "VirtualAllocEx",
114    "WriteProcessMemory",
115    "N.A.AdjustTokenPrivileges",
116    "N.A.AllocHGlobal",
117    "N.A.CreateDelegate",
118    "N.A.CreateProcess",
119    "N.A.CreateProcessWithLogonW",
120    "N.A.CreateThread",
121    "N.A.DuplicateToken",
122    "N.A.FindAmsiFun",
123    "N.A.FreeHGlobal",
124    "N.A.GetProcAddress",
125    "N.A.InitializeProcThreadAttributeList",
126    "N.A.LoadLibrary",
127    "N.A.LoadWithPartialName",
128    "N.A.OpenRemoteBaseKey",
129    "N.A.PostAsync",
130    "N.A.SetThreadToken",
131    "N.A.ShowDialog",
132    "N.A.ShowWindow",
133    "N.A.ShowWindowAsync",
134    "N.A.UpdateProcThreadAttribute",
135    "N.A.VirtualAlloc",
136    "N.A.VirtualProtect",
137    "N.A.WaitForSingleObject"
138 ],
139 "nativeCommands": [
140     "bitsadmin.exe",
141     "certutil.exe",

```

```

142     "cmd.exe",
143     "curl.exe",
144     "ipconfig",
145     "nslookup",
146     "powershell.exe",
147     "pwsh.exe",
148     "schtasks.exe",
149     "wget.exe",
150     "whoami"
151 ],
152 "noiseApis": [
153     "export-modulemember",
154     "import-localizeddata",
155     "import-module",
156     "set-item",
157     "set-location",
158     "set-strictmode",
159     "start-sleep",
160     "System.Management.Automation.PSScriptCmdlet.ShouldProcess",
161     "System.Threading.Monitor.Enter",
162     "System.Threading.Monitor.Exit",
163     "write-verbose"
164 ],
165 "noiseApiPrefixes": [
166     "System.Management.Automation.PSObject.add_",
167     "System.Management.Automation.Runspaces.",
168     "System.Management.Automation.SteppablePipeline.",
169     "System.Management.Automation.PSObject.SetToolTip",
170     "System.Management.Automation.PSObject.ReleaseMutex"
171 ],
172 "apiAliases": {
173     "cmd": "cmd.exe",
174     "ipconfig.exe": "ipconfig",
175     "nslookup.exe": "nslookup",
176     "powershell": "powershell.exe",
177     "pwsh": "pwsh.exe",
178     "ProcessStarter.StartProcess": "System.Diagnostics.Process.Start",
179     "System.Management.Automation.PSObject.CompileAssemblyFromSource": "CompileAssemblyFromSource",
180     "System.Management.Automation.PSObject.DownloadData": "System.Net.WebClient.DownloadData",
181     "System.Management.Automation.PSObject.DownloadFile": "System.Net.WebClient.DownloadFile",
182     "System.Management.Automation.PSObject.DownloadString": "System.Net.WebClient.DownloadString",
183     "System.Management.Automation.PSObject.Start": "System.Diagnostics.Process.Start",
184     "System.Management.Automation.PSObject.UploadData": "System.Net.WebClient.UploadData",
185     "System.Management.Automation.PSObject.UploadFile": "System.Net.WebClient.UploadFile",
186     "System.Management.Automation.PSObject.UploadString": "System.Net.WebClient.UploadString",
187     "System.Management.Automation.PSObject.UploadValues": "System.Net.WebClient.UploadValues",
188     "System.__ComObject.CopyHere": "Shell.Application.CopyHere",
189     "System.__ComObject.Namespace": "Shell.Application.Namespace",
190     "whoami.exe": "whoami",
191     "k32.func.OpenProcess": "OpenProcess",
192     "k32.func.VirtualAlloc": "VirtualAlloc",
193     "k32.func.VirtualAllocEx": "VirtualAllocEx",
194     "k32.func.WriteProcessMemory": "WriteProcessMemory",
195     "k32.func.CreateThread": "CreateThread",
196     "k32.func.CreateRemoteThread": "CreateRemoteThread",
197     "N.A.CreateDirectory": "System.IO.Directory.CreateDirectory",
198     "N.A.CreateProcess": "System.Diagnostics.Process.Start",
199     "N.A.DownloadData": "System.Net.WebClient.DownloadData",
200     "N.A.DownloadFile": "System.Net.WebClient.DownloadFile",
201     "N.A.DownloadString": "System.Net.WebClient.DownloadString",
202     "N.A.WriteAllBytes": "System.IO.File.WriteAllBytes",
203     "N.A.WriteAllLines": "System.IO.File.WriteAllLines",
204     "N.A.WriteAllText": "System.IO.File.WriteAllText",
205     "new-object:tcpclient": "System.Net.Sockets.TcpClient.Connect"
206 },
207 "capabilityNames": [
208     "CompileCSharp",
209     "ProcessSpawn",
210     "RemoteProcessOpen",
211     "RemoteMemoryAlloc",
212     "RemoteMemoryWrite",
213     "RemoteThreadCreate",
214     "ShellcodeInject",

```

```

215     "ScriptExec",
216     "Download",
217     "RegistryModify",
218     "FileWrite"
219 ],
220 "capabilityAliases": {
221     "add-type": "CompileCSharp",
222     "CompileAssemblyFromSource": "CompileCSharp",
223     "System.Management.Automation.PSObject.CompileAssemblyFromSource": "CompileCSharp",
224     "start-process": "ProcessSpawn",
225     "System.Diagnostics.Process.Start": "ProcessSpawn",
226     "ProcessStarter.StartProcess": "ProcessSpawn",
227     "N.A.CreateProcess": "ProcessSpawn",
228     "CreateProcess": "ProcessSpawn",
229     "CreateProcessWithLogonW": "ProcessSpawn",
230     "invoke-expression": "ScriptExec",
231     "iex": "ScriptExec",
232     "invoke-command": "ScriptExec",
233     "System.Management.Automation.CommandInvocationIntrinsics.NewScriptBlock": "ScriptExec",
234     "NewScriptBlock": "ScriptExec",
235     "System.Management.Automation.PowerShell.AddScript": "ScriptExec",
236     "AddScript": "ScriptExec",
237     "System.Management.Automation.PowerShell.Invoke": "ScriptExec",
238     "System.Management.Automation.PowerShell.BeginInvoke": "ScriptExec",
239     "ScriptBlock.Create": "ScriptExec",
240     "invoke-webrequest": "Download",
241     "invoke-restmethod": "Download",
242     "System.Net.WebClient.DownloadData": "Download",
243     "System.Net.WebClient.DownloadFile": "Download",
244     "System.Net.WebClient.DownloadString": "Download",
245     "curl.exe": "Download",
246     "wget.exe": "Download",
247     "bitsadmin.exe": "Download",
248     "certutil.exe": "Download",
249     "Microsoft.Win32.RegistryKey.SetValue": "RegistryModify",
250     "Microsoft.Win32.RegistryKey.CreateSubKey": "RegistryModify",
251     "Microsoft.Win32.RegistryKey.SetAccessControl": "RegistryModify",
252     "set-itemproperty": "RegistryModify",
253     "new-itemproperty": "RegistryModify",
254     "remove-itemproperty": "RegistryModify",
255     "System.IO.File.WriteAllBytes": "FileWrite",
256     "System.IO.File.WriteAllLines": "FileWrite",
257     "System.IO.File.WriteAllText": "FileWrite",
258     "add-content": "FileWrite",
259     "set-content": "FileWrite",
260     "out-file": "FileWrite",
261     "OpenProcess": "RemoteProcessOpen",
262     "k32.func.OpenProcess": "RemoteProcessOpen",
263     "VirtualAlloc": "RemoteMemoryAlloc",
264     "VirtualAllocEx": "RemoteMemoryAlloc",
265     "k32.func.VirtualAlloc": "RemoteMemoryAlloc",
266     "k32.func.VirtualAllocEx": "RemoteMemoryAlloc",
267     "N.A.VirtualAlloc": "RemoteMemoryAlloc",
268     "WriteProcessMemory": "RemoteMemoryWrite",
269     "k32.func.WriteProcessMemory": "RemoteMemoryWrite",
270     "System.Runtime.InteropServices.Marshal.Copy": "RemoteMemoryWrite",
271     "System.Runtime.InteropServices.Marshal.WriteByte": "RemoteMemoryWrite",
272     "CreateThread": "RemoteThreadCreate",
273     "CreateRemoteThread": "RemoteThreadCreate",
274     "k32.func.CreateThread": "RemoteThreadCreate",
275     "k32.func.CreateRemoteThread": "RemoteThreadCreate",
276     "N.A.CreateThread": "RemoteThreadCreate",
277     "Injector.Shellcode.Exec": "ShellcodeInject",
278     "Inject-Shellcode-Into-Remote-Proc": "ShellcodeInject"
279 }
280 }

```

S6 Full Residual Obfuscation-Burden Definition

We use a normalized residual metric to quantify how much visible obfuscation structure remains after deobfuscation. The metric is not intended to measure semantic correctness. Instead, it captures the residual wrapping, launching, and dynamic-execution burden preserved in the recovered output.

We do not directly use runtime instruction counts. For command-line-wrapped samples such as `powershell -enc ...`, a tool that fully peels the outer launcher may expose the embedded payload as explicit `ps1` code, whereas another tool may preserve the wrapper and implicitly defer later logic to a child process. In such cases, parent-process instruction counts reflect different execution boundaries and are no longer directly comparable.

S6.1 Feature Set

For each original sample x_0 and recovered output x_t , we reuse the six feature fields already collected in `results.json`:

- *wrapperDepth*: remaining outer wrapping depth.
- *launcherCount*: explicit launcher invocations such as `powershell`, `pwsh`, or `cmd`.
- *encodedCommandCount*: encoded command-line forms such as `-enc`.
- *dynamicExecPrimitiveCount*: unresolved dynamic-execution primitives; examples include `Invoke-Expression` and `ScriptBlock::Create`.
- *compilePrimitiveCount*: unresolved dynamic-compilation primitives.
- *byteArrayLiteralBytes*: numeric byte items preserved in literals such as `[byte[]](...)`.

S6.2 Per-Feature Residual Ratio

Let $v_f(x)$ denote the collected value of feature f on script x . We first apply the following feature transform:

$$g_f(v) = \begin{cases} \log_2(1 + \max(v, 0)), & \text{if } f = \text{byteArrayLiteralBytes}, \\ \max(v, 0), & \text{otherwise.} \end{cases}$$

The transform is used only to keep extremely large byte-array containers from dominating the final score. We then define the residual ratio of feature f between recovered output x_t and baseline x_0 as:

$$r_f(x_t, x_0) = \begin{cases} 0, & \text{if } g_f(v_f(x_0)) = 0 \text{ and } g_f(v_f(x_t)) = 0, \\ 1, & \text{if } g_f(v_f(x_0)) = 0 \text{ and } g_f(v_f(x_t)) > 0, \\ \min\left(\frac{g_f(v_f(x_t))}{g_f(v_f(x_0))}, 1\right), & \text{otherwise.} \end{cases}$$

This definition keeps every feature contribution within $[0, 1]$, treats the original sample as the reference upper bound, and avoids undefined ratios when the baseline feature value is zero.

S6.3 Sample-Level Residual Burden

We assign the following weights to the six features:

$$\begin{aligned} w_{\text{wrapperDepth}} &= 35, & w_{\text{launcherCount}} &= 35, \\ w_{\text{encodedCommandCount}} &= 10, & w_{\text{dynamicExecPrimitiveCount}} &= 8, \\ w_{\text{compilePrimitiveCount}} &= 8, & w_{\text{byteArrayLiteralBytes}} &= 4. \end{aligned}$$

The original weights follow a semantics-informed hierarchy based on how directly each residual feature obstructs access to the script’s executable intent. Table 4 gives the rationale for each feature.

Table 4: Rationale for the original residual-burden weights.

Feature	Weight	Rationale
<i>wrapperDepth</i>	35	An unresolved wrapper is a structural gate that can hide an entire downstream execution stage.
<i>launcherCount</i>	35	A remaining launcher hop prevents direct access to the next stage and may preserve a multi-stage execution chain.
<i>encodedCommandCount</i>	10	An encoded command directly conceals code, but is generally local and can often be decoded independently.
<i>dynamicExecPrimitiveCount</i>	8	Dynamic execution introduces runtime indirection, but a visible primitive does not necessarily represent unresolved obfuscation once its operand is explicit.
<i>compilePrimitiveCount</i>	8	Runtime compilation can conceal behavior, but an exposed compilation primitive with visible source may already reveal its intent.
<i>byteArrayLiteralBytes</i>	4	Byte-array volume is only a weak proxy for concealment and may increase when deobfuscation successfully exposes payload data.

Wrapper depth and launcher hops therefore receive 70% of the total weight because both act as structural gating mechanisms: leaving either unresolved may prevent an analyst from reaching all code behind that stage. Encoded commands receive a moderate weight because they hide code directly but are usually locally decodable. Dynamic-execution and compilation primitives receive lower weights because their presence alone does not establish that obfuscation remains when their operands are visible. Byte-array literals receive the lowest weight because exposed payload data should not be treated as strongly as an unresolved execution layer. The weights sum to 100 for direct interpretation as a percentage. They are a heuristic configuration rather than empirically learned or optimal parameters; the sensitivity analysis below evaluates five alternative configurations.

The residual obfuscation burden of a recovered output x_t relative to baseline x_0 is then defined as:

$$ResidualBurden(x_t, x_0) = 100 \cdot \frac{\sum_f w_f r_f(x_t, x_0)}{\sum_f w_f}.$$

Lower values indicate less residual obfuscation burden. A value of 100% means that the recovered output still preserves the full weighted burden of the original sample under this metric, while 0% means that none of the measured burden remains.

S6.4 Pairwise Aggregation

Different tools do not always produce valid outputs on the same set of samples. Therefore, we evaluate residual burden pairwise on common valid samples only. For a tool pair (a, b) , let $V_{a,b}$ denote the intersection of samples marked valid by both tools. The dataset-level residual burden reported for tool a is

$$\bar{B}_a = \frac{1}{|V_{a,b}|} \sum_{i \in V_{a,b}} ResidualBurden(x_i^a, x_i^0),$$

and the score for tool b is defined analogously. Lower values indicate better burden reduction.

S6.5 Weight-Sensitivity Analysis

To assess whether the reported comparison depends on the original heuristic weights, we recompute residual burden under five alternative schemes in addition to the original configuration. All values are calculated offline from the same inventory and post-deobfuscation outputs; no sample is re-executed. Within every scheme, the pairwise sample sets remain fixed: the IVD–PSDissect comparison contains 2,658 D-Script and 280 D-Cmdline common-valid samples, while the PowerPeeler–PSDissect comparison contains 3,407 D-Script and 374 D-Cmdline common-valid samples.

Table 5 lists the six evaluated configurations using the feature order introduced in Section S6.1. Equal weights remove the semantics-informed hierarchy; moderately balanced weights reduce its strength; and the final three schemes isolate structural, runtime-indirection, and encoding-related burden, respectively.

Table 5: Residual-burden weight schemes used in the sensitivity analysis.

Scheme	Wrap	Launch	Enc.	Dyn.	Comp.	Bytes
Original	35	35	10	8	8	4
Equal	1	1	1	1	1	1
Moderately balanced	26	26	13	12	12	11
Structural only	50	50	0	0	0	0
Runtime only	0	0	0	50	50	0
Encoding only	0	0	50	0	0	50

Tables 6 and 7 report the resulting residual burden. Bold values indicate the lower burden within each pair.

Table 6: D-Script residual burden under alternative weight schemes.

Scheme	IVD pair ($n = 2,658$)		PowerPeeler pair ($n = 3,407$)	
	IVD	PSDissect	PowerPeeler	PSDissect
Original	15.0694%	14.3288%	20.9553%	16.4130%
Equal	17.0731%	15.7941%	20.0359%	19.1369%
Moderately balanced	15.5375%	15.2767%	20.3566%	17.6059%
Structural only	13.8615%	11.6374%	21.1061%	13.7556%
Runtime only	34.0889%	30.1146%	32.8762%	37.2493%
Encoding only	3.4061%	5.4929%	6.1253%	6.4059%

Table 7: D-Cmdline residual burden under alternative weight schemes.

Scheme	IVD pair ($n = 280$)		PowerPeeler pair ($n = 374$)	
	IVD	PSDissect	PowerPeeler	PSDissect
Original	73.3893%	13.3488%	39.0077%	10.0980%
Equal	41.3690%	13.3532%	23.7566%	10.0862%
Moderately balanced	57.5429%	13.2345%	31.4173%	10.0071%
Structural only	99.1071%	12.9167%	51.0027%	9.8039%
Runtime only	24.8214%	24.6429%	18.6497%	18.5829%
Encoding only	0.3571%	2.3214%	1.6174%	1.8717%

The original, equal, moderately balanced, and structural-only schemes yield lower burden for PSDissect in every pair and dataset. Under runtime-only weighting, PSDissect remains lower in three of four comparisons, while PowerPeeler is lower on D-Script. Under encoding-only weighting, the baselines are lower in all four comparisons. These narrow schemes ignore wrappers and launcher chains and can penalize deobfuscators that expose byte-array payload data. Thus, PSDissect’s advantage is stable under broad or structurally motivated schemes, but is not asserted for metrics restricted to one residual-feature category.